

Fundamentals of Distributed Computing

1 Distributed Programs Overview

A distributed program is composed of a set of n asynchronous processes $p_1, p_2, \dots, p_n$ that communicate via message passing over a network. We assume that each process runs on a different processor.

Key Characteristics

- **Independent Processors:** Each process typically runs on its own isolated processor.
- **No Shared Memory:** Processes do not share global memory; they communicate solely via **message passing**.
- **Communication Channels (C_{ij}):** Denotes the channel from process p_i to process p_j . A message sent is denoted as m_{ij} .
- **Asynchronous Execution:** Processes may execute actions spontaneously. A sender does not wait for the delivery of a message to be complete (non-blocking).
- **Unpredictable Delays:** Message communication delay is finite but unpredictable.
- **No Global Clock:** There is no global clock instantaneously accessible to all processes.

2 Process Events and Linear Ordering

The execution of a process p_i produces a sequence of events $e_i^1, e_i^2, \dots, e_i^x$.

2.1 The Linear Order ($\rightarrow_i$)

The events at a process are linearly ordered by their occurrence. If we have two events such that:

$$e_i^1 \rightarrow_i e_i^2 \quad (1)$$

it means event 1 definitely happened before event 2 in the local timeline of p_i . You cannot, for example, post a second message (e_i^2) before the first one (e_i^1).

2.2 The Sequence (h_i)

The symbol h_i represents the **History** of process p_i . It is the set of all events produced by that process. The process execution is denoted by:

$$i = \langle h_i, \rightarrow_i \rangle \quad (2)$$

Because this is a linear order, there is a clear, non-ambiguous sequence of actions for each individual process.

2.3 Causal Dependencies

The relation $\rightarrow_i$ defines the causal dependencies among the events of p_i . In a single process, the "linear order" ensures that the cause always happens before the effect. For example, if you assign $x = 5$ (Event A) and then `print(x)` (Event B), Event B depends on Event A:

$$Event_A \rightarrow_i Event_B \quad (3)$$

3 Causal Dependency via Message Exchange

While the linear order $\rightarrow_i$ handles events within a single process, the relation $\rightarrow_{msg}$ captures the causal dependency created when two different processes interact.

3.1 The Message Relation ($\rightarrow_{msg}$)

For every message m that is exchanged between two processes, the sending of the message must logically occur before its receipt. This is defined as:

$$send(m) \rightarrow_{msg} rec(m) \quad (4)$$

3.2 Key Implications

The relation $\rightarrow_{msg}$ is fundamental for understanding how information flows through a network:

- **Causal Link:** The event $rec(m)$ is causally dependent on $send(m)$. You cannot receive what has not yet been sent.
- **Synchronization:** This relation provides a way to "stitch together" the individual histories (h_i, h_j) of different processes.
- **Directionality:** Unlike local events which are ordered by a physical sequence on one processor, $\rightarrow_{msg}$ represents the flow of information across the communication channel C_{ij} .

$$\text{If } e_i = send(m) \text{ and } e_j = rec(m), \text{ then } e_i \text{ is the cause of } e_j. \quad (5)$$

4 Space-Time Diagrams

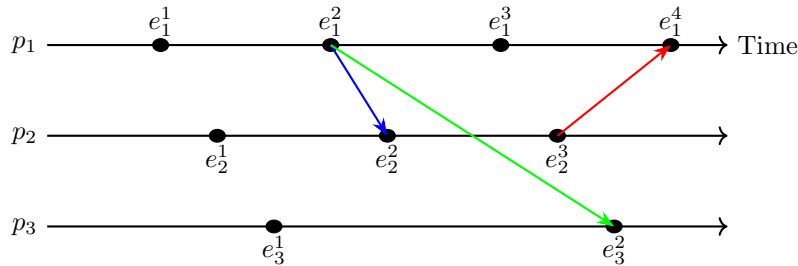
The evolution of a distributed execution is visually represented by a **space-time diagram**. This diagram allows us to track local event ordering and message-based causal dependencies simultaneously.

4.1 Visual Components

- **Horizontal Lines:** Represent the progress of a process (p_i) over time.
- **Dots:** Represent **atomic events**. These are considered indivisible and instantaneous.
- **Slant Arrows:** Represent message transfers. The tail is the *send* event, and the head is the *receive* event.

4.2 Diagram Representation

Below is a space-time diagram illustrating the interaction between three processes (p_1, p_2, p_3):



4.3 Analysis of Process p_1

In the figure above, we can classify the events for process p_1 :

1. e_1^1 : Initial event.
2. e_1^2 : A **message send event** (triggering the blue and green arrows).
3. e_1^3 : An **internal event** (no external communication).
4. e_1^4 : A **message receive event** (from p_2).

5 Physical Constraints: Clock Drift

In theory, we assume asynchronous processes; in practice, these processes run on hardware with physical clocks that inevitably diverge.

5.1 Definition of Clock Drift

Every computer contains a physical oscillator. Due to imperfections, temperature, and aging, these oscillators do not tick at the exact same frequency.

- **Drift**: The cumulative difference in time between two physical clocks.
- **Impact**: Even with periodic synchronization (like NTP), two servers can disagree on the exact timestamp of an event at the microsecond or millisecond level.

5.2 The Flash Sale Problem

In a highly contended system (e.g., a Flash Sale at 12:00 PM), Clock Drift creates a sequencing paradox:

1. Server A (Fast) logs a request at 12:00:00.001.
2. Server B (Slow) logs a request at 11:59:59.999.

If the system relies solely on physical timestamps, the request at Server B might be rejected as “too early,” even if it physically arrived at the same time as the request at Server A.

5.3 Conclusion: Logic Over Physics

Because physical clocks are unreliable for fine-grained ordering, distributed systems rely on the **Happened-Before relation** ($\rightarrow$) to establish a logical sequence that remains consistent regardless of the underlying clock drift.

6 Formal Model of Clock Synchronization

The software clock $C_p(t)$ is coupled to hardware via interrupts, where its rate of change is determined by the oscillator frequency.

6.1 Bounded Drift Rate

We define the maximum drift rate as ρ . For a clock to stay within specifications, its derivative must be bounded:

$$1 - \rho \leq \frac{dC_p(t)}{dt} \leq 1 + \rho \quad (6)$$

Because one machine may be slightly “fast” while another is “slow,” perfect agreement on exact physical time is impossible in a distributed system.

6.2 Synchronization Metrics

There are two primary metrics for evaluating clock synchronization:

- **Precision (π):** The goal of *internal synchronization*. We aim to keep the deviation between any two machines (p, q) within a specific bound:

$$\forall t, \forall p, q : |C_p(t) - C_q(t)| \leq \pi \quad (7)$$

- **Accuracy (α):** The goal of *external synchronization*. We aim to keep the machine's clock bound to a standard time (UTC):

$$\forall t, \forall p : |C_p(t) - t| \leq \alpha \quad (8)$$

The Verdict

Because of hardware drift, physical clocks alone cannot determine the exact sequence of events across a network. We must shift the focus from “what time is it?” to “in what order did this happen?” using logical clocks.

7 Case Study: Quick Commerce and Logical Ordering

7.1 The Real Problem: No Single Reliable Clock

In systems like BigBasket, Blinkit, or UPI payment networks, multiple distributed services handle a single user action.

Example: Ordering the Last Loaf of Bread

When a customer places an order, the following events may occur almost simultaneously on different machines:

- **P1:** Inventory Server decrements stock.
- **P2:** Order Service processes the request.
- **P3:** Dispatch Service assigns delivery.

If two customers order the very last loaf at roughly the same physical time, the system must decide:

- Whose order happened first?
- Which order sees “Out of Stock”?

Because of **Clock Drift**, relying on physical timestamps is unsafe. Two servers may disagree on the exact time of arrival. Therefore, we need **logical ordering**, not physical time.

7.2 Lamport Logical Clocks

Lamport proposed that instead of synchronizing physical clocks, we assign each process a simple counter called a **Logical Clock**.

Each process p_i maintains a counter C_i .

Update Rules

1. **Internal Event:** Increment clock.

$$C_i \leftarrow C_i + 1 \quad (9)$$

2. **Send Event:** Increment clock and attach timestamp.

$$C_i \leftarrow C_i + 1 \quad (10)$$

Message carries timestamp $ts = C_i$.

3. **Receive Event:** Synchronize then increment.

$$C_j \leftarrow \max(C_j, ts) + 1 \quad (11)$$

Example Execution

Assume:

- Inventory Server ($P1$) has $C_1 = 5$.
- It updates stock $\Rightarrow C_1 = 6$.
- It sends message to Order Service with timestamp 6.
- Order Service ($P2$) has $C_2 = 4$.
- On receiving message:

$$C_2 = \max(4, 6) + 1 = 7 \quad (12)$$

This guarantees:

$$e_i \rightarrow e_j \implies C(e_i) < C(e_j) \quad (13)$$

Thus, if one event causally influences another, its logical timestamp is strictly smaller.

7.3 Why This Works

Lamport clocks capture the **Happened-Before Relation** ($\rightarrow$):

- If $a \rightarrow b$, then $C(a) < C(b)$.
- If $C(a) < C(b)$, it does **not necessarily** imply $a \rightarrow b$ (logical clocks do not detect concurrency).

They provide a **consistent causal ordering** even when physical clocks disagree.

7.4 Total Ordering for Global Consistency

In replicated systems (e.g., multiple dark stores updating shared inventory), we may require **Total Ordering**.

If two events receive the same logical timestamp, we break ties using process IDs:

$$(C_i, i) < (C_j, j) \quad (14)$$

This ensures all servers apply updates in the **exact same sequence**, preventing inconsistencies like:

- One store believing stock exists
- Another marking it sold out

7.5 Connection to UPI Payments

When a UPI transaction shows “Payment Pending,” multiple distributed components (sender bank, receiver bank, NPCI switch) must agree on the final status.

Even with concurrent global transactions:

- The system must ensure exactly one final outcome.
- All parties must observe events in a consistent order.

This agreement is achieved through distributed consensus and ordering protocols built on causal reasoning similar to Lamport’s logical clocks.

Key Insight

In distributed systems, we cannot trust physical time. We trust causality.

7.6 Achieving Total Order: Breaking Ties

Logical clocks provide a **partial order** because they only guarantee:

$$a \rightarrow b \implies C(a) < C(b) \quad (15)$$

However, two concurrent events may receive the **same logical timestamp**. To construct a **Total Order** ($<$) — where every pair of events is comparable — we introduce a deterministic tie-breaker.

The Two Ingredients of Total Order

- **Logical Timestamp (C):** The Lamport clock value.
- **Process ID (i):** A unique identifier for each process.

Definition of Total Order

Let event a occur at process p_i with timestamp $C(a)$, and event b occur at process p_j with timestamp $C(b)$.

We define:

$$a < b \iff \begin{cases} C(a) < C(b), \\ C(a) = C(b) \text{ and } i < j \end{cases} \quad \text{or} \quad (16)$$

Thus, we compare ordered pairs:

$$(C(a), i) < (C(b), j) \quad (17)$$

This creates a **lexicographic ordering**.

Why This Works

- If $a \rightarrow b$, then $C(a) < C(b)$, so $a < b$.
- If two events are concurrent and receive the same timestamp, the process ID ensures a consistent global decision.
- Every pair of events becomes comparable.

Therefore, the total order $<$ is:

- **Consistent with causality**
- **Deterministic**
- **Identical at all processes**

Practical Importance

Total ordering is crucial in:

- Replicated databases
- Inventory management systems
- Distributed logs
- Payment processing systems

It ensures that all processes apply updates in the **exact same sequence**, even when events are concurrent.

8 Causal Precedence Relation (The “Happened-Before” Relation)

The execution of a distributed application results in a set of distributed events produced by all processes. Let $H = \bigcup_i h_i$ denote the set of all events executed across the entire system.

Meaning of the Symbol $\sim$ For example:

$$e_i^x$$

means

- i = process index
- x = event number at that process

Thus e_2^3 means:

the third event executed by process p_2 .

8.1 Formal Definition

We define a binary relation on the set H , denoted as $\rightarrow$, which expresses causal dependencies between events. For any two events e_i^x and e_j^y in H , the relation $e_i^x \rightarrow e_j^y$ is true if and only if it satisfies one of the following conditions:

$$\forall e_i^x, e_j^y \in H : e_i^x \rightarrow e_j^y \iff \begin{cases} e_i^x \rightarrow_i e_j^y & \text{if } i = j \text{ and } x < y \\ e_i^x \rightarrow_{msg} e_j^y & \text{(Message exchange)} \\ \exists e_k^z \in H : e_i^x \rightarrow e_k^z \wedge e_k^z \rightarrow e_j^y & \text{(Transitivity)} \end{cases} \quad (18)$$

8.2 Key Concepts

- **Irreflexive Partial Order:** The relation induces a partial order $(H, \rightarrow)$. It is “partial” because not every pair of events can be compared. If no path exists between two events, they are considered *concurrent*.
- **Lamport’s Relation:** This is the famous “happens-before” relation. If $e_i \rightarrow e_j$, then e_j is either directly or transitively dependent on e_i .
- **Graphical Interpretation:** In a space-time diagram, $e_i \rightarrow e_j$ means there is a continuous path moving forward in time made of:
 1. Horizontal process-line segments.
 2. Slanted message arrows.

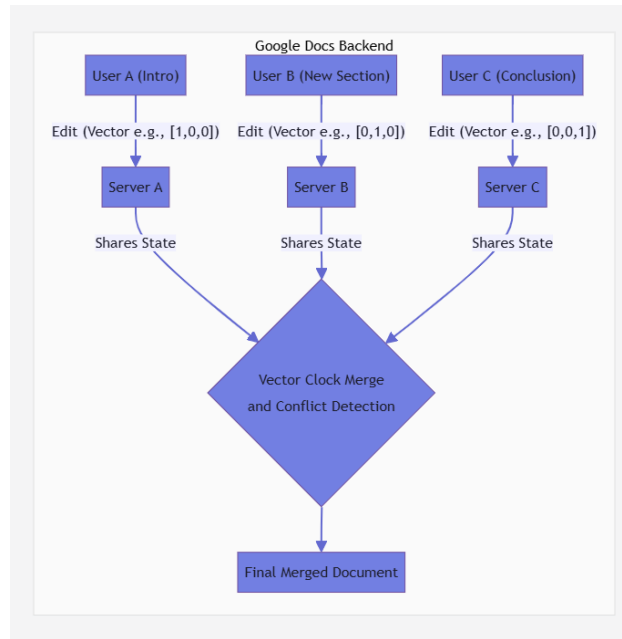
Note: If you can trace a path from event A to event B following the arrows and the lines, then $A \rightarrow B$.

9 Vector Clocks: Capturing True Causality

While Lamport clocks establish a sequence, they cannot distinguish between causal dependency and coincidence. **Vector Clocks** allow a system to detect concurrency ($\parallel$) by maintaining an array of counters.

9.1 The Collaborative Editing Example (Google Docs)

Imagine three users (A, B, C) editing a document simultaneously. A Vector Clock is an array where each index i corresponds to the local clock of process P_i .



- **Local Edit:** User A edits the intro. Their vector changes from $[0, 0, 0]$ to $[1, 0, 0]$.
- **Synchronization:** When User A sends their state to User B (who is at $[0, 2, 0]$), User B merges the information to become $[1, 2, 0]$, then increments their own index to $[1, 3, 0]$.
- **Conflict Detection:** If User A has $[1, 0, 0]$ and User B has $[0, 1, 0]$, neither is "greater" than the other. The system immediately knows these edits are **concurrent** and requires a merge strategy.

10 Advanced Vector Clock Scenarios

10.1 Thought Experiment 1: Conflict Detection in Collaborative Editing

When two users perform contradictory actions (e.g., User A deletes a sentence while User B italicizes it), the system determines concurrency via the *partial order* of their vectors.

- If $V_A = [1, 0]$ and $V_B = [0, 1]$, then $V_A \parallel V_B$.
- The system detects that neither user had seen the other's update at the time of their own edit.
- **Information Needed:** The system requires the full vector to know that the events were independent, triggering a *merge strategy* rather than a simple overwrite.

10.2 Thought Experiment 2: The Element-wise Merge

Merging vectors (taking the max of each index) allows for **transitive knowledge**.

If P_A with $[5, 2, 1]$ receives $[4, 7, 0]$ from P_B , the new state is $[5, 7, 1]$.

This is critical because P_A now knows P_B has interacted with P_C and seen 7 of its events. This prevents P_A from accepting an older, stale message from P_C later on, as it now knows a more recent state exists.

10.3 Thought Experiment 3: Preventing Phantom Items (Flipkart/Myntra)

In a “1 Left in Stock” scenario, Vector Clocks prevent double-selling.

- **Concurrency Check:** If two “Buy Now” requests arrive with concurrent vectors ($V_1 \parallel V_2$), the inventory service knows a race condition occurred.
- **Deterministic Tie-breaking:** The system uses a secondary rule (e.g., lower Process ID or a higher priority user tier) to pick a winner.
- **Causality:** This ensures the second user’s request is logically placed *after* the first, seeing a “Sold Out” status.

10.4 Formal Rules for Vector Clocks (VC)

For a system of n processes, each process P_i maintains a vector $VC_i[1 \dots n]$.

1. **Internal Event:** P_i increments its own entry:

$$VC_i[i] := VC_i[i] + 1 \quad (19)$$

2. **Message Passing:** When P_i sends a message, it attaches its entire vector VC_i .
3. **Receive Event:** When P_j receives a message with vector V_{msg} , it updates its vector using an element-wise maximum:

$$\forall k : VC_j[k] := \max(VC_j[k], V_{msg}[k]) \quad (20)$$

Then, it increments its own entry $VC_j[j]$.

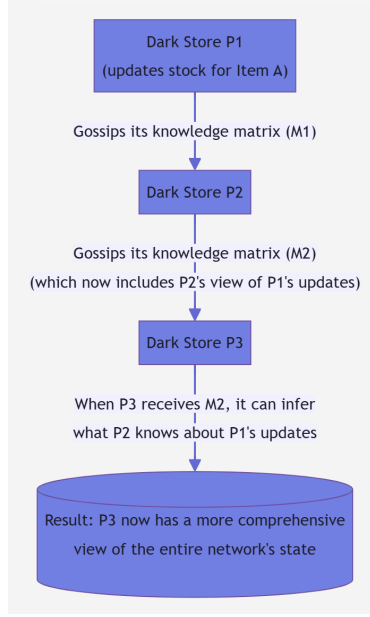
10.5 Comparing Vector Clocks

- $V_1 \leq V_2 \iff \forall i : V_1[i] \leq V_2[i]$
- $V_1 < V_2 \iff (V_1 \leq V_2) \wedge (\exists j : V_1[j] < V_2[j])$
- **Concurrency:** $V_1 \parallel V_2 \iff \neg(V_1 < V_2) \wedge \neg(V_2 < V_1)$

Analysis: The Phantom Item Problem

In the Flipkart “Only 1 Left” scenario, Vector Clocks prevent phantom sales by detecting if two “Buy Now” clicks were concurrent.

- If $VC(User1) \parallel VC(User2)$, the inventory server detects a race condition.
- The system then uses a deterministic tie-breaker (like the lowest Process ID) to grant the item to one user and notify the other that the item is sold out, ensuring the database never enters an inconsistent state.



11 Matrix Clocks: Knowledge of Global Knowledge

A **Matrix Clock** is an extension of Vector Clocks that allows processes to track not only the progress of others but also the *mutual knowledge* between all participants.

11.1 Formal Representation

For a system of n processes, each process P_i maintains an $n \times n$ matrix M_i :

- $M_i[i, j]$ is the j -th element of P_i 's vector clock (what P_i knows about P_j).
- $M_i[j, k]$ represents P_i 's knowledge of what P_j knows about P_k .

11.2 The Update Rules

When P_i sends a message to P_j , it attaches its **entire matrix** M_i . Upon receiving a matrix W from P_i , process P_j performs:

1. **Vector Update:** $\forall k : M_j[j, k] := \max(M_j[j, k], W[i, k])$
2. **Matrix Update:** $\forall k, m : M_j[k, m] := \max(M_j[k, m], W[k, m])$

11.3 Case Study: Inventory Reconciliation (Blinkit)

Matrix Clocks are essential for background synchronization tasks, such as **Anti-Entropy** and **Garbage Collection**.

- **Knowledge of Knowledge:** If Dark Store P_3 sees that $M_3[2, 1] \geq X$, it knows that Store P_2 has already received all updates from P_1 up to time X .
- **Efficiency:** If P_3 receives a gossip from P_2 containing an update from P_1 , it checks its matrix. If $M_3[3, 1]$ is already greater than the incoming update's timestamp, P_3 discards it immediately without re-processing, preventing redundant work.

12 Concurrency and Independent Events

In a distributed execution, not all events are related by cause and effect. Some events happen independently.

12.1 Non-Causal Relation ($\parallel$)

For any two events e_i and e_j , $e_i e_j$ denotes that event e_j is **not** directly or transitively dependent on event e_i .

- Event e_i does not causally affect e_j .
- Event e_j is completely unaware of the execution of e_i .

12.2 Concurrent Events ($\parallel$)

If two events have **no causal path** between them in either direction, they are said to be **concurrent**.

$$e_i \parallel e_j \iff (e_i e_j) \wedge (e_j e_i) \quad (21)$$

12.3 Properties of Concurrency

- **Symmetry:** If $e_i \parallel e_j$, then $e_j \parallel e_i$.
- **Non-Transitivity:** Concurrency is **not** transitive. It is possible that $(e_i \parallel e_j)$ and $(e_j \parallel e_k)$, yet $e_i \rightarrow e_k$.
- **The Trichotomy:** For any two events e_i and e_j in a distributed execution, exactly one of the following must be true:
 1. $e_i \rightarrow e_j$ (e_i caused e_j)
 2. $e_j \rightarrow e_i$ (e_j caused e_i)
 3. $e_i \parallel e_j$ (They are independent/concurrent)

13 Logical vs. Physical Concurrency

In a distributed system, it is essential to distinguish between events that are independent and events that happen at the same global "wall-clock" time. Understanding this distinction is the key to reasoning about distributed computations.

13.1 Definitions

- **Logical Concurrency ($e_i \parallel e_j$):** Two events are logically concurrent if and only if they do not causally affect each other. This is a property of the **program logic** and causal structure.

$$e_i \parallel e_j \iff (e_i e_j) \wedge (e_j e_i) \quad (22)$$

- **Physical Concurrency:** This occurs when events happen at the exact same instant in **physical time** (wall-clock time). This depends entirely on hardware oscillators and is often unreliable due to clock drift and network delays.

13.2 Key Insight: Logic over Physics

Logically concurrent events do **not** need to occur at the same physical time to be considered "simultaneous" by the system.

- Whether independent events occur at the same time or in a specific physical order does not change the **outcome** of the computation.
- **The Verdict:** In distributed systems, *causality (logic) is more important than the clock (physics)*. For all theoretical and practical purposes, we treat logically concurrent events as if they occurred at the same instant.

"The correctness of a distributed computation depends on causal dependencies, not on the precise physical time at which events occur."

13.3 Case Study 1: Food Delivery Logistics (Swiggy/Zomato)

Consider a system managing a food delivery app with two processes: P_{chef} (Restaurant) and P_{rider} (Delivery Partner).

- **Event e_1 (10:00:01 AM):** The chef marks the order as "Food Ready" in Delhi.
- **Event e_2 (10:00:04 AM):** A rider in Bangalore updates their GPS coordinates.

Analysis:

1. **Physical Perspective:** These events are **not** physically concurrent as they occurred 3 seconds apart.
2. **Logical Perspective:** These are **logically concurrent** ($e_1 \parallel e_2$). There is no message exchange or shared state between the chef in Delhi and the rider in Bangalore. The system outcome remains identical regardless of which event is logged first.

13.4 Case Study 2: Independent E-commerce Transactions (Amazon)

Suppose two customers purchase unrelated products:

- **Customer A:** Buys headphones in Mumbai.
- **Customer B:** Buys shoes in Chennai.

Even if these requests reach the servers at slightly different physical times, they are **logically concurrent** because:

- They do not modify the same inventory record.
- There is no causal dependency between the two buyers.
- The order in which the database processes these events does not affect the correctness of the system state.

13.5 Conclusion: Why this Matters

Distributed systems focus on **causal ordering** because physical clocks cannot be perfectly synchronized. By identifying logically concurrent events (like $\{e_1^3, e_2^4, e_3^3\}$), the system gains the flexibility to process them in any order that is computationally efficient without risking data corruption.

14 Models of Communication Networks

Communication channels in distributed systems may provide different guarantees about the order in which messages are delivered. The three common models are:

- **Non-FIFO**
- **FIFO (First-In First-Out)**
- **Causal Ordering (CO)**

These models differ in how message ordering is preserved during delivery.

14.1 Non-FIFO Model

In the **Non-FIFO model**, a communication channel behaves like a *set*. Messages sent by a process may be received in any arbitrary order.

If a process sends messages

$$m_1, m_2, m_3$$

the receiver may obtain them as:

$$m_3, m_1, m_2$$

Thus, no ordering guarantees exist.

Example: In a food delivery platform, a restaurant server may send:

- m_1 : “Food is ready”
- m_2 : “Assign rider”

Without ordering guarantees, the delivery service may receive m_2 before m_1 , producing an inconsistent state.

14.2 FIFO Model

In the **FIFO model**, each channel behaves like a queue. Messages sent by a process to another process are received in the same order in which they were sent.

If process p_i sends:

$$send(m_1) \rightarrow send(m_2)$$

then process p_j must receive:

$$rec(m_1) \rightarrow rec(m_2)$$

FIFO ordering is guaranteed only for messages sent along the same channel.

Example: In an order management system, a restaurant service sends:

- m_1 : “Order accepted”
- m_2 : “Food prepared”

FIFO guarantees the order service receives the messages in the same order.

14.3 Causal Ordering Model

The **Causal Ordering (CO) model** is based on Lamport's *happens-before* relation ($\rightarrow$). It ensures that causally related messages are delivered in an order consistent with their causal dependency.

Causal Ordering Property:

For any two messages m_{ij} and m_{kj} :

$$send(m_{ij}) \rightarrow send(m_{kj}) \Rightarrow rec(m_{ij}) \rightarrow rec(m_{kj}) \quad (23)$$

Thus, if one message causally precedes another, the receiver must observe them in the same order.

14.4 Example: Missing Context Problem

Consider three processes:

- p_1 : Manager
- p_2 : Employee
- p_3 : Database

1. p_1 sends message m_1 : "Update project deadline to Friday".
2. p_2 receives m_1 and sends m_2 : "Acknowledged deadline" to p_3 .

Without causal ordering, p_3 may receive m_2 before m_1 , which is inconsistent. Causal ordering guarantees:

$$rec(m_1) \rightarrow rec(m_2)$$

ensuring that the database observes the update before the acknowledgement.

14.5 Relation Between the Models

The ordering guarantees satisfy the hierarchy:

$$CO \subset FIFO \subset Non-FIFO \quad (24)$$

- **Non-FIFO:** No ordering guarantees.
- **FIFO:** Preserves order between a sender and receiver.
- **Causal Ordering:** Preserves logical cause-effect relationships across processes.

14.6 Importance in Distributed Systems

Causal ordering simplifies distributed algorithm design because it preserves the logical dependencies between events.

For example, in **replicated database systems**, updates must be applied in the same causal order at all replicas to maintain consistency. Without causal ordering, each update must be manually checked for missing dependencies. With causal ordering, such checks become unnecessary because the communication system already guarantees correct ordering.

15 Global State of a Distributed System

The **global state (GS)** of a distributed system is the collection of the **local states of all processes** and the **states of all communication channels**. It represents a system-wide *snapshot* of the distributed execution.

15.1 Local State of a Process

The **local state** of a process consists of the contents of its:

- processor registers
- stack and local memory
- program variables and buffers

Let

$$LS_i^x$$

denote the state of process p_i **after event e_i^x and before event e_i^{x+1}** .

- LS_i^0 : initial state of process p_i
- LS_i^x : state produced after executing the first x events

Thus the execution of a process evolves as

$$LS_i^0 \rightarrow LS_i^1 \rightarrow LS_i^2 \rightarrow \dots$$

15.2 Send and Receive Notation

We write

$$send(m) \leq LS_i^x$$

to denote that process p_i has already executed the send event of message m before reaching state LS_i^x . Formally, this means:

$$\exists y \leq x : e_i^y = send(m)$$

Similarly,

$$rec(m) \leq LS_j^y$$

means that process p_j has already received message m before reaching state LS_j^y .

15.3 Channel State

Communication channels may contain messages that have been sent but not yet received. Such messages are called **messages in transit**.

The state of channel C_{ij} is defined as

$$SC_{ij}^{xy} = \{m_{ij} \mid send(m_{ij}) \leq LS_i^x \wedge rec(m_{ij}) \leq LS_j^y\}$$

Interpretation:

A message m_{ij} belongs to the channel state if:

- it was sent by p_i before state LS_i^x
- but has not yet been received by p_j before state LS_j^y

Thus SC_{ij}^{xy} represents all **messages currently in transit** on channel C_{ij} .

15.4 Global State Definition

The **global state** is defined as

$$GS = \{LS_1^{x_1}, LS_2^{x_2}, \dots, LS_n^{x_n}, SC_{ij}\}$$

It contains:

- the local state of every process
- the state of every communication channel

Conceptually, this represents a **snapshot of the entire distributed system**.

15.5 The Global Clock Problem

Ideally, a global snapshot would record all process states at the same instant. However, in distributed systems:

- there is **no perfectly synchronized global clock**
- processes execute asynchronously
- messages experience unpredictable delays

Therefore snapshots cannot be taken at exactly the same physical time. Instead, snapshots must satisfy **causality constraints**.

15.6 Consistency of Global States

A global state is **consistent** if it does not violate causal relationships.

Consistency condition:

$$rec(m_{ij}) \leq LS_j^y \Rightarrow send(m_{ij}) \leq LS_i^x$$

This ensures that:

An effect cannot appear without its cause.

In other words, if a snapshot records that a message was received, it must also record that the message was sent.

15.7 Types of Global States

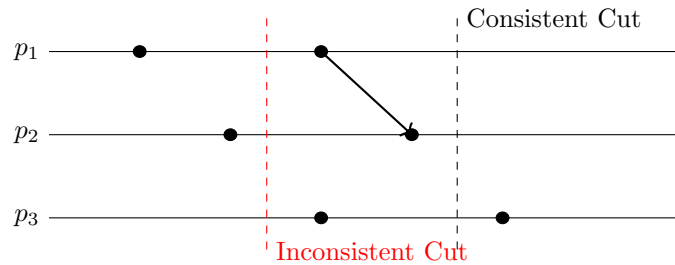
- **Consistent State**
Every received message has its corresponding send event recorded.
- **Inconsistent State**
A message appears as received but its send event is missing. Such states violate causality and cannot occur in real executions.
- **Transitless State**
All communication channels are empty:

$$\forall i, j \quad SC_{ij} =$$

- **Strongly Consistent State**
A state that is both **consistent** and **transitless**.

15.8 Space–Time Illustration

Global states are often visualized using a **space–time diagram**. Each horizontal line represents a process timeline and dots represent events.



A vertical **cut** across the diagram selects one local state from each process. If the cut includes a receive event without its corresponding send event, the resulting global state is **inconsistent**.

15.9 Example: Food Delivery Platform

Consider a distributed system with:

- p_1 : Restaurant service
- p_2 : Order management
- p_3 : Delivery assignment

Suppose p_1 sends message m_{12} : “Food Ready” to p_2 .

- If a snapshot records that p_2 has received m_{12} but p_1 has not recorded sending it, the state is **inconsistent**.
- A **consistent snapshot** ensures that if the receive event exists, the corresponding send event also exists.

15.10 Applications

Recording global states is fundamental for:

- debugging distributed programs
- monitoring system behavior
- deadlock detection
- checkpointing and recovery
- verification of distributed algorithms

The most widely used algorithm for capturing consistent snapshots is the **Chandy–Lamport Snapshot Algorithm**.

16 Cuts of a Distributed Computation

In a **space–time diagram** of a distributed system, each horizontal line represents the timeline of a process p_i , and each dot on the line represents an event e_i^x executed by that process. Time progresses from left to right.

Definition of a Cut A **cut** is a zigzag line drawn across the diagram that intersects each process timeline at exactly one point. The cut divides the entire execution into two regions:

- **PAST**: all events that lie to the *left* of the cut
- **FUTURE**: all events that lie to the *right* of the cut

For a cut C , we denote:

$PAST(C)$ = set of all events before the cut

$FUTURE(C)$ = set of all events after the cut

Thus, a cut partitions the distributed execution into events that *have already happened* and events that *have not yet happened* with respect to that snapshot.

Cuts and Global State Every cut corresponds to a **global state** of the distributed system. Intuitively, the cut represents the moment at which we observe the state of each process.

If $e_i^{\max(PAST(C))}$ denotes the latest event of process p_i that lies in the PAST of cut C , then the global state represented by cut C consists of:

- the local state of each process after its latest event in $PAST(C)$
- the set of messages that were sent in the PAST but received in the FUTURE

Formally, the global state corresponding to cut C is:

$$GS = \left\{ LS_i^{\max(PAST_i(C))}, SC_{jk} \right\}$$

where the channel state is

$$SC_{jk} = \{m \mid send(m) \in PAST(C) \wedge rec(m) \in FUTURE(C)\}$$

This means that a message belongs to the channel state if:

- it was **sent before the cut**, and
- it is **received after the cut**

Such messages are called **messages in transit**.

Consistent Cut A cut is called **consistent** if it does not violate causality.

Formally, a cut C is consistent if:

$$rec(m) \in PAST(C) \Rightarrow send(m) \in PAST(C)$$

This means that if the cut contains a receive event, it must also contain the corresponding send event. In other words:

A consistent cut never records an *effect without its cause*.

All messages that cross a consistent cut must go from

$$PAST(C) \rightarrow FUTURE(C)$$

These represent messages currently in transit.

Inconsistent Cut A cut is **inconsistent** if a message crosses the cut in the opposite direction:

$$FUTURE(C) \rightarrow PAST(C)$$

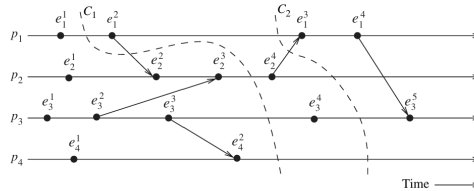
This corresponds to the situation where the cut contains a receive event but does not contain the corresponding send event.

Such a state violates causality and therefore cannot represent a valid global state of the distributed computation.

46

A model of distributed computations

Figure 2.3 Illustration of cuts in a distributed execution.



Cuts in a space-time diagram provide a powerful graphical aid in representing and reasoning about global states of a computation.

Explanation of Figure 2.3 In the diagram there are four processes:

$$p_1, p_2, p_3, p_4$$

Each dot labeled e_i^x represents the x^{th} event executed by process p_i .

Arrows between processes represent **message transmissions**.

Two cuts are shown:

- C_1
- C_2

Cut C_1 (Inconsistent Cut)

Cut C_1 intersects the diagram in such a way that a message is received in the PAST while its send event lies in the FUTURE.

This means the snapshot contains the effect (receive) without the cause (send), which violates causality. Therefore, C_1 corresponds to an **inconsistent global state**.

Cut C_2 (Consistent Cut)

Cut C_2 satisfies the causality condition:

- every message received in the PAST was also sent in the PAST
- messages crossing the cut go only from PAST to FUTURE

Thus, C_2 corresponds to a **consistent global state**.

Messages crossing C_2 represent **messages in transit**.

Key Insight Cuts provide a powerful graphical method for reasoning about distributed systems because they allow us to visualize **global states** of the computation even though the processes execute asynchronously and no global clock exists.

17 Past and Future Cones of an Event

In a distributed computation, events influence each other through the **happened-before relation** ($\rightarrow$). For any event e_j , we can identify:

- all events that **could have influenced** e_j
- all events that **could be influenced by** e_j

These sets form the **past cone** and **future cone** of the event.

Past of an Event

An event e_j could only have been affected by events e_i such that

$$e_i \rightarrow e_j$$

That is, there exists a causal chain from e_i to e_j .

The **past of** e_j is therefore defined as

$$Past(e_j) = \{e_i \in H \mid e_i \rightarrow e_j\}$$

where

- H is the set of all events in the distributed computation
- $\rightarrow$ is the happened-before relation

Thus $Past(e_j)$ contains all events that **causally influence** e_j .

Graphically, these events form the **past cone** of e_j in the space-time diagram.

Past of e_j on a Specific Process

Let

$$Past_i(e_j)$$

denote the subset of $Past(e_j)$ that occurred on process p_i .

Since events on a process are totally ordered by the local order relation $\rightarrow_i$, the set $Past_i(e_j)$ is totally ordered.

The **latest event** in this set is denoted by

$$\max(Past_i(e_j))$$

This represents the most recent event on process p_i that could have affected e_j .

In distributed computations, this event is always a **message send event**, because information from p_i can influence another process only through message transmission.

Surface of the Past Cone

Define

$$Max_Past(e_j) = \{\max(Past_i(e_j)) \mid \forall i\}$$

This set contains the latest event on every process that causally influenced e_j .

This set forms the **surface of the past cone** of e_j .

Important property:

- $Max_Past(e_j)$ forms a **consistent cut** of the computation.

This means that the boundary separating the past from the rest of the computation respects causality.

Future of an Event

The **future of** e_j consists of all events that are causally affected by e_j .

Formally,

$$Future(e_j) = \{e_i \in H \mid e_j \rightarrow e_i\}$$

Thus $Future(e_j)$ contains all events that **depend on the occurrence of** e_j .

These events form the **future cone** of e_j .

Future of e_j on a Specific Process

Let

$$Future_i(e_j)$$

be the subset of $Future(e_j)$ that occurs on process p_i .

The earliest event in this set is

$$\min(Future_i(e_j))$$

This represents the **first event on process p_i that is affected by** e_j .

Such an event is always a **message receive event**, because a process can be affected by another process only after receiving a message from it.

Surface of the Future Cone

Define

$$Min_Future(e_j) = \{\min(Future_i(e_j)) \mid \forall i\}$$

This set represents the earliest event on each process that is causally affected by e_j .

It forms the **surface of the future cone** of e_j and also corresponds to a **consistent cut**.

Events Concurrent with e_j

Events that lie:

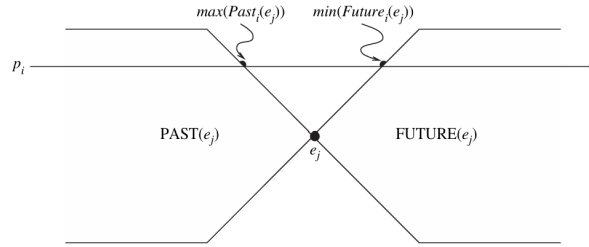
- after $\max(Past_i(e_j))$
- before $\min(Future_i(e_j))$

on process p_i are **concurrent** with e_j .

Formally, the set of events concurrent with e_j is

$$H - Past(e_j) - Future(e_j)$$

These events neither influence e_j nor are influenced by e_j .



Explanation of Figure

In the diagram:

- The horizontal line represents the timeline of process p_i .
- The central event is e_j .
- The region to the **left** of e_j represents $Past(e_j)$.
- The region to the **right** of e_j represents $Future(e_j)$.

The two boundary points labeled

$$\max(Past_i(e_j))$$

and

$$\min(Future_i(e_j))$$

represent:

- the last event on process p_i that could affect e_j
- the first event on process p_i that could be affected by e_j

All events between these two boundaries are **concurrent with** e_j .

The past and future cones visually resemble a **light cone** in physics: information flows outward from the event e_j through message chains, forming the future cone, while the past cone contains all events whose information could have reached e_j .

17.1 Case Study: Food Delivery Platform (Explaining Global State, Cuts, and Cones)

Consider a distributed food delivery platform similar to *Swiggy* or *Zomato*. The system consists of three independent services that communicate via messages:

- p_1 : Restaurant Service
- p_2 : Order Management Service
- p_3 : Delivery Assignment Service

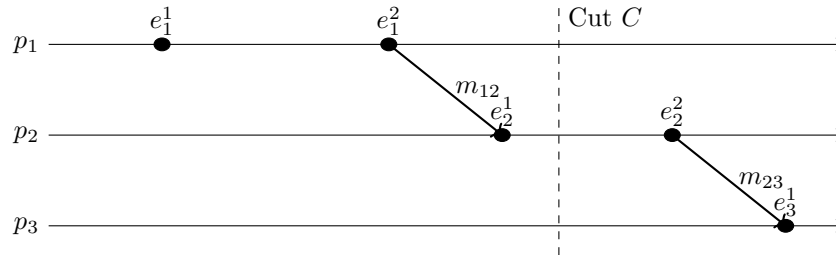
Each service runs on a separate machine and processes events asynchronously.

Scenario

A customer places an order. The following sequence of events occurs:

- e_1^1 : Restaurant receives the order.
- e_1^2 : Restaurant sends message m_{12} : “Food Ready” to Order Service.
- e_2^1 : Order Service receives m_{12} .
- e_2^2 : Order Service sends message m_{23} : “Assign Delivery” to Delivery Service.
- e_3^1 : Delivery Service receives m_{23} and assigns a rider.

Space–Time Representation



Understanding the Global State

The dashed vertical line represents a **cut**. This cut selects a **local state** from each process:

- Restaurant state after e_1^2
- Order service state after e_2^1
- Delivery service state before e_3^1

Thus the global state consists of:

- Local state of Restaurant
- Local state of Order Service

- Local state of Delivery Service
- Messages currently in transit

In this example, message m_{23} may be in transit if it was sent but not yet received.

Consistent vs Inconsistent Cut

A cut is **consistent** if every received message has its send event included in the past of the cut.

In the example:

- m_{12} is received only after it was sent.
- The cut respects the causal order.

Thus the cut corresponds to a **valid global state**.

If a cut included the receive event of m_{12} but excluded its send event, the cut would be **inconsistent**.

Past Cone of an Event

Suppose we focus on event e_3^1 (delivery assignment).

The **past cone** of e_3^1 includes all events that causally affected it:

- e_2^2 (order service sends delivery request)
- e_2^1 (order service receives restaurant message)
- e_1^2 (restaurant sends “food ready”)
- e_1^1 (restaurant receives order)

These events form the causal history of e_3^1 .

Future Cone of an Event

If the rider assignment event e_3^1 triggers another event, such as:

- e_3^2 : Rider starts delivery
- e_2^3 : Order status updated

these events belong to the **future cone** of e_3^1 .

Concurrent Events

Events that are neither in the past nor in the future of e_3^1 are called **concurrent**.

For example:

- another restaurant preparing a different order
- a separate delivery service assigning a rider for another order

These events occur independently and do not influence e_3^1 .

Key Insight

Cuts and cones help visualize distributed executions:

- **Cuts** represent global snapshots of the system.
- **Past cones** represent causal history.
- **Future cones** represent causal impact.
- Events outside both cones are **concurrent**.

These concepts are fundamental in distributed algorithms such as the **Chandy–Lamport snapshot algorithm**, which records consistent global states.

18 Models of Process Communications

Processes in a distributed system communicate by exchanging messages. Two fundamental models describe how this communication occurs:

- **Synchronous Communication**
- **Asynchronous Communication**

These models differ in whether the sender must wait for the receiver while sending a message.

18.1 Synchronous Communication Model

The **synchronous communication model** is a **blocking** model.

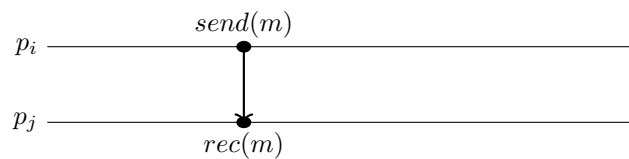
When a process sends a message, the sender process is blocked until the receiver has accepted the message. In other words, both the sender and receiver must synchronize during message exchange.

Formally, if process p_i sends a message m to process p_j :

- p_i executes $send(m)$
- p_i waits until p_j performs $rec(m)$
- only after the message is received does p_i resume execution

Thus message communication behaves like a **handshake** between the two processes.

Illustration



Here the sender and receiver synchronize at the same logical moment.

Real Example Consider a payment authorization system:

- A payment service sends a request to the bank.
- The payment service must wait for the bank to accept the request before proceeding.

This type of communication is synchronous because both systems must coordinate during the transaction.

Advantages

- Easier to design and reason about distributed algorithms
- Simpler implementation
- Reduced nondeterminism

Disadvantages

- Lower parallelism
- Frequent blocking of processes
- Higher risk of deadlocks

18.2 Asynchronous Communication Model

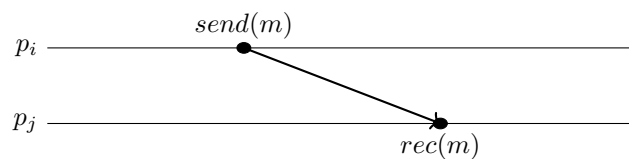
The **asynchronous communication model** is a **non-blocking** communication model.

After sending a message, the sender process continues execution without waiting for the receiver.

- The message is stored in a **buffer**
- The receiver collects the message when it is ready

Thus the sender and receiver do not need to synchronize during message exchange.

Illustration



Here the sender continues execution while the message is still in transit.

Real Example Most modern distributed systems use asynchronous communication.

Example: A food delivery platform

- Restaurant service sends “Order Ready” to delivery service.
- The restaurant immediately continues preparing other orders.
- The delivery service processes the message later.

The sender does not wait for the receiver.

Buffering Since messages may arrive faster than they are processed, systems store them temporarily in **buffers**.

However, if many messages arrive quickly, a **buffer overflow** may occur.

Advantages

- Higher parallelism
- Better system performance
- Processes can execute concurrently

Disadvantages

- More complex buffer management
- Higher nondeterminism
- Harder to design and verify distributed algorithms
- Larger state space of possible executions

18.3 Comparison of the Two Models

Feature	Synchronous	Asynchronous
Blocking	Yes	No
Synchronization	Required	Not required
Parallelism	Lower	Higher
Implementation complexity	Simpler	More complex
Deadlock risk	Higher	Lower

Key Insight

Neither communication model is universally superior.

- **Synchronous communication** simplifies reasoning about distributed algorithms.
- **Asynchronous communication** provides higher concurrency and better performance.

Modern distributed systems often implement asynchronous communication while using protocols to manage ordering, buffering, and reliability.

Chapter Glimpse: Key Insights

- **Distributed System Model**
 - System consists of processes $p_1, p_2, \dots, p_n$ communicating via message passing.
 - No shared memory, no global clock, unpredictable message delays.
 - Computation is asynchronous.
- **Events and Local Order**
 - Events of process p_i : $e_i^1, e_i^2, \dots$
 - Local order: $e_i^x \rightarrow_i e_i^y$ if $x < y$
 - Process history: h_i ; global event set: $H = \bigcup_i h_i$
- **Causal Dependency**
 - Message rule: $send(m) \rightarrow rec(m)$
 - Combined with local order and transitivity forms Lamport's **happened-before relation** ($\rightarrow$).
 - $(H, \rightarrow)$ forms a partial order of events.
- **Concurrency**
 - Events are concurrent if:
$$e_i \parallel e_j \iff (e_i e_j) \wedge (e_j e_i)$$
 - Concurrent events are independent and may execute in any order.

- **Logical vs Physical Time**

- Physical clocks suffer from drift and cannot ensure correct ordering.
- Distributed systems rely on **logical time** and causality.

- **Logical Clock Mechanisms**

- **Lamport Clocks**: scalar timestamps satisfying

$$e_i \rightarrow e_j \Rightarrow C(e_i) < C(e_j)$$

- **Vector Clocks**: detect causal relation and concurrency.
- **Matrix Clocks**: track knowledge of knowledge between processes.

- **Communication Ordering Models**

$$CO \subset FIFO \subset Non-FIFO$$

- Non-FIFO: no ordering guarantees.
- FIFO: preserves order between sender and receiver.
- Causal Ordering: preserves cause-effect relationships.

- **Global State**

- Snapshot of the distributed system:

$$GS = \{LS_i^{x_i}, SC_{ij}\}$$

- Channel state contains **messages in transit**.
- Consistency condition:

$$rec(m) \Rightarrow send(m)$$

- **Cuts**

- A cut divides execution into $PAST(C)$ and $FUTURE(C)$.
- Represents a **global snapshot**.
- Consistent cut:

$$rec(m) \in PAST(C) \Rightarrow send(m) \in PAST(C)$$

- **Past and Future Cones**

- Past of event:

$$Past(e_j) = \{e_i \mid e_i \rightarrow e_j\}$$

- Future of event:

$$Future(e_j) = \{e_i \mid e_j \rightarrow e_i\}$$

- Concurrent events:

$$H - Past(e_j) - Future(e_j)$$

- **Process Communication Models**

- **Synchronous**: sender blocks until receiver accepts message.
- **Asynchronous**: sender continues execution after sending.
- Most real systems use asynchronous communication for scalability.

- **Core Principle**

Correctness in distributed systems depends on **causality**, not physical time.

19 Framework for a System of Logical Clocks

Logical clocks provide a way to assign timestamps to events in a distributed system when physical clocks cannot reliably determine the order of events.

19.1 Basic Idea

In distributed systems there is no global clock. Therefore, instead of relying on physical time, we assign each event a **logical timestamp** that captures the causal ordering between events.

A **system of logical clocks** consists of:

- a **time domain** T
- a **logical clock function** C

19.2 Time Domain

The time domain T is a set of logical time values.

- Elements of T are ordered using the relation $<$
- This ordering represents the **causal precedence** between events
- It is analogous to the “earlier than” relation in physical time

Thus logical time provides an ordering similar to physical time, but based purely on **causal relationships**.

19.3 Logical Clock Function

The logical clock is defined as a function

$$C : H \rightarrow T$$

where

- H = set of all events in the distributed system
 - T = logical time domain
 - $C(e)$ = logical timestamp assigned to event e
- Thus each event in the system is mapped to a logical time value.

19.4 Clock Consistency Condition

A logical clock must respect the causal ordering of events.

If event e_i causally precedes event e_j , then the timestamp of e_i must be smaller than the timestamp of e_j .

Formally:

$$e_i \rightarrow e_j \Rightarrow C(e_i) < C(e_j)$$

This property is called the **Clock Consistency Condition**.

Interpretation

- Causes must always appear before their effects.
- Logical timestamps must increase along causal chains.
- Logical clocks preserve the ordering implied by the *happened-before* relation.

19.5 Strong Clock Consistency

A stronger condition may also hold:

$$e_i \rightarrow e_j \iff C(e_i) < C(e_j)$$

This means:

- If e_i happens before e_j , then $C(e_i) < C(e_j)$.
- If $C(e_i) < C(e_j)$, then e_i must causally precede e_j .

When this bidirectional condition holds, the logical clock system is said to be **strongly consistent**.

Key Insight

- Logical clocks provide a way to order events without synchronized physical clocks.
- They preserve the causal structure of distributed computations.
- Lamport clocks satisfy the clock consistency condition but are not strongly consistent because they cannot detect concurrency.

19.6 Implementing Logical Clocks

To implement a system of logical clocks in a distributed system, two key aspects must be defined:

- the **data structures** used to represent logical time
- the **protocol rules** used to update these clocks

These mechanisms ensure that logical timestamps satisfy the **clock consistency condition**:

$$e_i \rightarrow e_j \Rightarrow C(e_i) < C(e_j)$$

Logical Clock Data Structures

Each process p_i maintains local data structures that represent its logical notion of time.

- **Local Logical Clock (lc_i)**
 - Measures the progress of process p_i .
 - Updated whenever the process executes an event.
 - Represents the process's **local logical time**.
- **Global Logical Clock View (gc_i)**
 - Represents process p_i 's **local view of the global logical time**.
 - Used to assign timestamps to events in a way that remains consistent with other processes.
 - Usually the local clock lc_i is included as part of gc_i .

Thus every process maintains its own estimate of the system's logical time.

Logical Clock Protocol

To maintain consistency across processes, logical clocks are updated using two fundamental rules.

Rule R1: Local Clock Update This rule defines how the local logical clock lc_i is updated when process p_i executes an event.

Events include:

- internal computation events
- message send events
- message receive events

The clock is updated to ensure that logical time always moves forward.

Rule R2: Message-Based Clock Update This rule defines how logical time information is communicated between processes.

- Logical timestamp information is **piggybacked** on messages.
- When a message is received, the receiving process updates its logical clock using the timestamp contained in the message.
- This allows processes to maintain a consistent view of global progress.

Properties of Logical Clock Systems

All logical clock systems implement rules $R1$ and $R2$, ensuring that:

- logical time always increases along causal chains
- timestamps respect the happened-before relation
- the monotonicity property of logical clocks is preserved

Different logical clock systems vary in:

- how logical time is represented
- how timestamps are updated and propagated

Examples include:

- **Lamport Clocks (Scalar Time)**
- **Vector Clocks**
- **Matrix Clocks**

Each provides additional capabilities such as detecting concurrency or tracking causal dependencies more precisely.

20 Scalar Time (Lamport Logical Clocks)

Scalar time was introduced by **Lamport (1978)** to assign logical timestamps to events and to help establish a **total ordering** of events in a distributed system.

20.1 Basic Idea

In scalar time representation:

- The **time domain** T is the set of non-negative integers.
- Each process p_i maintains a single integer clock:

$$C_i$$

- This clock represents both:
 - the local logical clock of the process
 - its view of global logical time

Thus logical time is represented using a **single scalar value**.

20.2 Clock Update Rules

Lamport clocks follow two rules corresponding to the logical clock framework.

Rule R1: Local Event Rule

Before executing any event (internal, send, or receive), process p_i updates its clock as:

$$C_i = C_i + d \quad (d > 0)$$

Typically:

$$d = 1$$

because:

- it uniquely orders events within a process
- it keeps timestamps small

Thus every event increases the local logical time.

Rule R2: Message Rule

When process p_i sends a message:

- it piggybacks its current clock value C_i with the message

When process p_j receives a message with timestamp C_{msg} :

1. Update clock:

$$C_j = \max(C_j, C_{msg})$$

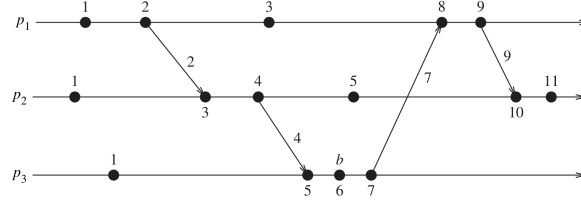
2. Execute Rule R1:

$$C_j = C_j + 1$$

3. Deliver the message.

This ensures that logical time always moves forward across processes.

Explanation of Scalar Time Evolution



The figure illustrates the evolution of **Lamport scalar clocks** in a distributed system with three processes p_1, p_2 , and p_3 .

Each process maintains a logical clock C_i that increases for every event according to Rule *R1*:

$$C_i = C_i + 1$$

When a process sends a message, it piggybacks its current clock value. Upon receiving a message with timestamp C_{msg} , the receiving process updates its clock using Rule *R2*:

$$C_j = \max(C_j, C_{msg}) + 1$$

Thus message reception may cause a **jump in logical time** to maintain causal ordering. For example:

- When p_2 receives a message from p_1 with timestamp 2, its clock updates from 1 to 3.
- When p_3 receives a message from p_2 with timestamp 4, its clock updates to 5.
- When p_1 later receives a message from p_3 with timestamp 7, its clock jumps to 8.

The timestamps therefore satisfy the causality condition:

$$e_i \rightarrow e_j \Rightarrow C(e_i) < C(e_j)$$

However, scalar clocks are **not strongly consistent**. Even if $C(e_i) < C(e_j)$, it does not necessarily mean that $e_i \rightarrow e_j$; the events may simply be concurrent.

20.3 Consistency Property

Scalar clocks satisfy the **clock consistency condition**:

$$e_i \rightarrow e_j \Rightarrow C(e_i) < C(e_j)$$

Thus causal ordering is preserved.

However, the reverse is not necessarily true.

20.4 Total Ordering of Events

Scalar timestamps can be used to totally order events.

Problem:

- Two events at different processes may have the same timestamp.

Example:

$$C(e_1) = C(e_2)$$

To break ties, events are ordered using process identifiers.
Each event timestamp becomes:

$$(t, i)$$

where

- t = logical timestamp
- i = process identifier

The total order relation is defined as:

$$x \prec y \iff t_x < t_y \quad \text{or} \quad (t_x = t_y \wedge i_x < i_y)$$

Thus events are ordered lexicographically.

20.5 Event Counting Property

If the increment value is always $d = 1$, scalar time has an interesting property.

If an event e has timestamp h , then:

$$h - 1$$

represents the minimum number of events that must have occurred before e along a causal chain.

This value is called the **height** of event e .

Thus scalar clocks also provide an estimate of the **logical length of the causal path**.

20.6 Limitation: No Strong Consistency

Scalar clocks are **not strongly consistent**.

Although

$$e_i \rightarrow e_j \Rightarrow C(e_i) < C(e_j)$$

the reverse does not hold:

$$C(e_i) < C(e_j) \not\Rightarrow e_i \rightarrow e_j$$

This means timestamps alone cannot determine causal relationships.

20.7 Reason for This Limitation

Scalar clocks combine:

- local logical time
 - global logical view
- into a single integer value.

Because of this compression, information about **which process caused which event** is lost.

Therefore scalar clocks cannot distinguish between:

- causally related events
- concurrent events

Key Insight

- Lamport clocks preserve causal ordering.
- They allow total ordering of events.
- However, they cannot detect concurrency.

This limitation motivated the development of **vector clocks**, which capture full causal information.

21 Vector Time

21.1 Definition

Vector clocks were independently proposed by **Fidge**, **Mattern**, and **Schmuck** to capture the full causal relationship between events in a distributed system.

In this system, logical time is represented by an n -dimensional vector of non-negative integers.

Each process p_i maintains a vector clock:

$$VT_i = (vt_i[1], vt_i[2], \dots, vt_i[n])$$

where:

- $vt_i[i]$: local logical clock of process p_i
- $vt_i[j]$: p_i 's latest knowledge of process p_j 's logical time

If

$$vt_i[j] = x$$

then process p_i knows that process p_j has executed at least x events.

Thus the vector clock VT_i represents process p_i 's **view of global logical time**.

21.2 Clock Update Rules

Each process updates its vector clock using two rules.

Rule R1 (Local Event)

Before executing an event at process p_i :

$$vt_i[i] = vt_i[i] + d \quad (d > 0)$$

Typically:

$$d = 1$$

Rule R2 (Message Event)

When process p_i sends a message, it piggybacks its vector clock VT_i .

When process p_j receives message (m, VT) :

1. Update vector clock:

$$vt_j[k] = \max(vt_j[k], VT[k]) \quad \forall k$$

2. Execute Rule R1:

$$vt_j[j] = vt_j[j] + 1$$

3. Deliver the message.

21.3 Timestamp of an Event

The timestamp of an event is simply the vector clock value of the process at the moment the event occurs.

$$C(e) = VT_i$$

Initial Condition

Initially all vector clocks start at:

$$(0, 0, \dots, 0)$$

for all processes.

Explanation of Vector Clock Evolution

The figure illustrates the evolution of **vector clocks** in a distributed system with three processes:

$$p_1, p_2, p_3$$

Each process maintains a vector clock:

$$VT_i = [vt_i[1], vt_i[2], vt_i[3]]$$

where:

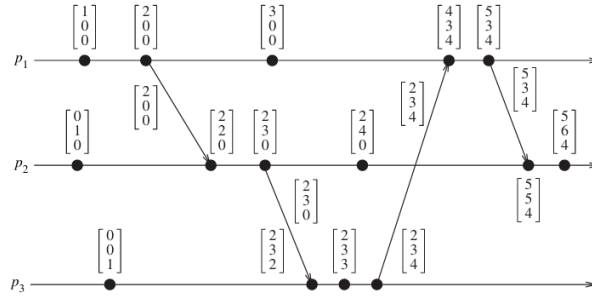
- $vt_i[i]$ represents the local logical time of process p_i
- $vt_i[j]$ represents process p_i 's knowledge of the progress of process p_j

Initially all vector clocks are:

$$[0, 0, 0]$$

The figure shows how vector timestamps propagate through message exchanges and capture the **causal dependencies** between events across processes.

The figure shows the evolution of vector clocks for three processes p_1 , p_2 , and p_3 . Each event is labeled with its vector timestamp $[v_1, v_2, v_3]$, representing the knowledge of events at processes p_1 , p_2 , and p_3 respectively.



Example: Process p_1 The first event at p_1 increments its own component:

$$[1, 0, 0]$$

The second event becomes:

$$[2, 0, 0]$$

When p_1 later receives a message from p_3 carrying timestamp $[2, 3, 4]$, it updates its vector clock using the element-wise maximum:

$$\max([3, 0, 0], [2, 3, 4]) = [3, 3, 4]$$

Then it increments its own component to produce:

$$[4, 3, 4]$$

Thus the vector clock at p_1 records that:

- p_1 has executed 4 events,
- it knows that p_2 has executed 3 events,
- and p_3 has executed 4 events.

This demonstrates how vector clocks capture the causal knowledge propagated through message exchanges.

21.4 Comparison of Vector Timestamps

Let v_h and v_k be two vector timestamps.

$$v_h = v_k \iff \forall x \ v_h[x] = v_k[x]$$

- Both vectors are identical in every component.

$$v_h \leq v_k \iff \forall x \ v_h[x] \leq v_k[x]$$

- Every component of v_h is less than or equal to the corresponding component of v_k .

$$v_h < v_k \iff (v_h \leq v_k) \wedge (\exists x \ v_h[x] < v_k[x])$$

- v_h is strictly smaller in at least one component. This implies the event with timestamp v_h **happened before** the event with timestamp v_k .

$$v_h \parallel v_k \iff \neg(v_h < v_k) \wedge \neg(v_k < v_h)$$

- Neither vector dominates the other. The corresponding events are **concurrent**.

21.5 Basic Properties of Vector Clocks

Isomorphism Recall that the **happened-before relation** ($\rightarrow$) defines a **partial order** on the set of events in a distributed computation.

When events are timestamped using vector clocks, the ordering between events is preserved exactly by their vector timestamps.

If events x and y have vector timestamps v_h and v_k respectively,

$$x \rightarrow y \iff v_h < v_k$$

$$x \parallel y \iff v_h \parallel v_k$$

Thus the causal structure of events and their vector timestamps have a **one-to-one correspondence**. This property is called an **isomorphism**. It means that vector timestamps capture the exact causal relationships between events in the distributed execution.

Simplified Timestamp Comparison If we know the processes where the events occurred, comparison becomes simpler.

Suppose:

- event x occurs at process p_i
- event y occurs at process p_j
- timestamps are v_h and v_k

Then:

$$x \rightarrow y \iff v_h[i] \leq v_k[i]$$

$$x \parallel y \iff v_h[i] > v_k[i] \wedge v_h[j] < v_k[j]$$

This allows faster detection of causal relationships when the originating processes of the events are known.

Simplified Comparison of Vector Timestamps Assume event x occurs at process p_i and event y occurs at process p_j . Let their vector timestamps be v_h and v_k respectively.

$$x \rightarrow y \iff v_h[i] \leq v_k[i]$$

This means that the vector timestamp of y contains at least as much information about process p_i as the timestamp of x . Thus event y must have occurred after learning about the event x .

$$x \parallel y \iff v_h[i] > v_k[i] \wedge v_h[j] < v_k[j]$$

This means each process has knowledge that the other does not have. Therefore neither event causally precedes the other and the events are **concurrent**.

Strong Consistency Vector clocks satisfy the **strong consistency condition**:

$$x \rightarrow y \iff V(x) < V(y)$$

This means that by comparing vector timestamps we can determine exactly whether two events are causally related.

However, for this property to hold, the vector clock must have dimension equal to the number of processes n in the system.

Event Counting If the increment value in rule $R1$ is always $d = 1$:

- $vt_i[i]$ represents the number of events that have occurred at process p_i .
- If an event e has timestamp v_h , then $v_h[j]$ represents the number of events at process p_j that causally precede e .

Thus vector clocks encode not only ordering but also the **causal event count** across processes.

Applications Because vector clocks capture causal dependencies precisely, they are used in:

- distributed debugging
- causal message ordering
- causal distributed shared memory
- determining checkpoint consistency
- optimistic recovery algorithms

21.6 On the Size of Vector Clocks

A natural question is whether a distributed computation with n processes always requires vector clocks of size n .

A vector clock stores the latest known logical time of every process. Thus, if the goal is to explicitly track the progress of every process, a vector clock of size n is required.

However, if vector clocks are used only to determine causality between events, a smaller clock may sometimes suffice.

Partial Order and Dimension The happened-before relation ($\rightarrow$) induces a **partial order** on the set of events.

The **dimension of a partial order** is the minimum number of *linear extensions* whose intersection produces exactly the original partial order.

A **linear extension** is a total ordering of events that respects all causal dependencies.

The minimum clock size required to determine causality is equal to the dimension of this partial order.

Example: Client–Server Interaction Consider two processes:

- P_1 : client
- P_2 : server

Requests and responses occur in strict alternating order:

$$send \rightarrow receive \rightarrow send \rightarrow receive$$

All events are totally ordered, so the dimension of the partial order is 1. Therefore a **scalar clock** is sufficient.

Example: Concurrent Sends Consider two processes P_1 and P_2 such that each sends a message to the other before receiving the other's message.

The two send events are concurrent.

A scalar clock cannot represent this concurrency correctly, so a vector clock of size 2 is required.

Crown Structure In certain executions messages form a cyclic dependency called a **crown**. In such cases the dimension of the partial order equals the number of processes n , and vector clocks of size n are necessary to determine causal relationships.

21.7 Example: Dimension of a Partial Order

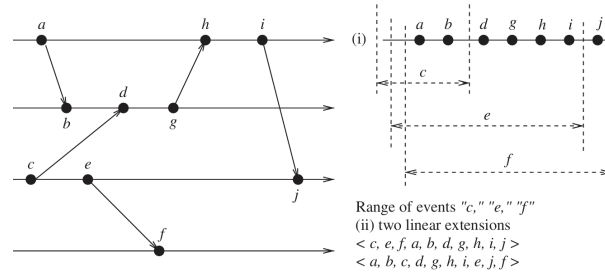


Figure shows an execution involving four processes. Events are connected by message arrows representing the *happened-before* relation.

Longest Causal Chain One of the longest chains in the execution is:

$$a \rightarrow b \rightarrow d \rightarrow g \rightarrow h \rightarrow i \rightarrow j$$

These events must always appear in this order in any valid execution.

Events Outside the Chain The events c , e , and f are not part of this chain. They are only partially constrained by causality and therefore can occur at multiple positions relative to the chain.

The right side of the figure illustrates the earliest and latest positions where these events can appear.

Linear Extensions A **linear extension** is a total ordering of events that respects all causal dependencies. Two valid linear extensions for this execution are:

$$\lambda_1 = \langle c, e, f, a, b, d, g, h, i, j \rangle$$

$$\lambda_2 = \langle a, b, c, d, g, h, i, e, j, f \rangle$$

Both orders maintain the causal relationships defined by the partial order.

Dimension of the Partial Order The dimension of a partial order is the minimum number of linear extensions whose intersection produces exactly the original partial order.

In this example, the intersection of λ_1 and λ_2 reconstructs the causal relations among the events. Therefore the dimension of the partial order is:

$$\text{dimension} = 2$$

Implication Although the execution involves four processes, only two dimensions are sufficient to capture the causal structure of the execution.

However, determining this dimension is computationally difficult, and therefore practical systems usually use vector clocks of size equal to the number of processes.

22 Efficient Implementations of Vector Clocks

When the number of processes in a distributed system is large, vector clocks become expensive because every message must carry a vector of size n .

Thus the message overhead grows linearly with the number of processes.

Observation Between two successive messages sent from process p_i to process p_j , only a few entries of the vector clock usually change.

This is because only a small subset of processes interact frequently.

Differential Technique Singhal and Kshemkalyani proposed a technique where a process sends only those entries of the vector clock that have changed since the last message sent to the same destination process.

If entries $i_1, i_2, \dots, i_k$ have changed to values $v_1, v_2, \dots, v_k$, then the sender piggybacks the compressed timestamp

$$(i_1, v_1), (i_2, v_2), \dots, (i_k, v_k)$$

instead of the entire vector.

Upon receiving the message, process p_j updates its clock as

$$VT_j[i_k] = \max(VT_j[i_k], v_k)$$

Auxiliary Vectors To determine which entries have changed, each process maintains two additional vectors:

- $LS_i[j]$ (Last Sent): value of $VT_i[i]$ when p_i last sent a message to p_j .
- $LU_i[j]$ (Last Update): value of $VT_i[i]$ when entry $VT_i[j]$ was last updated.

An entry k is included in the message if

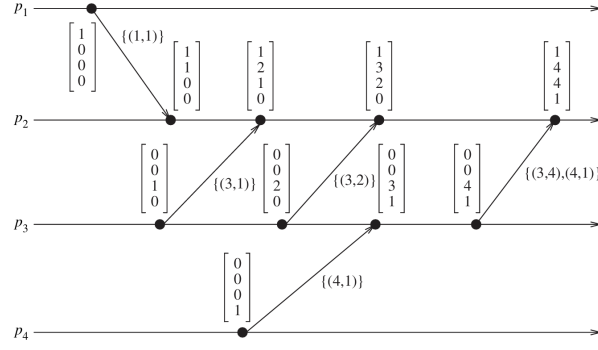
$$LS_i[j] < LU_i[k]$$

This means that the entry changed after the last message sent to p_j .

Advantage This technique reduces message size, communication bandwidth, and buffer storage requirements while still maintaining the correct vector clock semantics.

22.1 Example of Singhal–Kshemkalyani Differential Technique

Figure illustrates the differential technique for efficiently transmitting vector clocks. The system consists of four processes p_1, p_2, p_3 , and p_4 . Each event shows the current vector clock at that process, while the messages carry only the entries of the vector clock that have changed since the last message to that destination.



Initial Communication Process p_1 sends a message to p_2 . At the sending event the vector clock of p_1 is:

$$[1, 0, 0, 0]^T$$

Since this is the first message to p_2 , the tuple

$$\{(1, 1)\}$$

is piggybacked. This indicates that the first component (process p_1) has value 1. After receiving the message, process p_2 updates its vector clock accordingly.

Message from p_3 to p_2 Later, process p_3 sends a message to p_2 with compressed timestamp

$$\{(3, 1)\}$$

This indicates that the third component of the vector clock has changed and its new value is 1. Thus process p_2 updates the third entry of its vector clock.

Second Message from p_3 to p_2 When p_3 sends another message to p_2 , only the third component has changed from 1 to 2. Therefore the message carries:

$$\{(3, 2)\}$$

This informs p_2 that the logical time of process p_3 has advanced to 2 since the last communication between the two processes.

Message from p_4 Process p_4 sends a message containing

$$\{(4, 1)\}$$

indicating that the fourth component of the vector clock has value 1. Process p_2 updates the fourth entry accordingly.

Final Message In the last communication, the message contains two tuples:

$$\{(3, 4), (4, 1)\}$$

This indicates that both the third and fourth components have changed since the previous message, and therefore both updates must be applied.

Key Idea Instead of transmitting the entire vector clock of size n with every message, only the entries that have changed since the last message to that destination are transmitted. This significantly reduces message size, communication bandwidth, and storage overhead.

This optimization is particularly useful in large distributed systems where only a small subset of processes interact frequently.

22.2 Fowler–Zwaenepoel Direct Dependency Technique

The Fowler–Zwaenepoel technique reduces the communication overhead of vector clocks by transmitting only a **scalar value** with each message. Instead of maintaining full vector clocks during execution, each process tracks only its **direct dependencies**. The complete causal information can later be reconstructed *off-line* by recursively following these direct dependencies.

Dependency Vector Each process p_i maintains a dependency vector D_i of size n .

$D_i[j]$ = sequence number of the latest event on p_j that directly influences p_i

Initially,

$$D_i[j] = 0 \quad \forall j$$

Update Rules The dependency vector is updated using the following rules:

1. Whenever an event occurs at process p_i :

$$D_i[i] = D_i[i] + 1$$

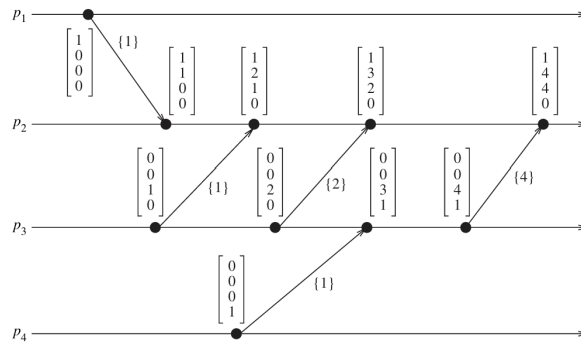
This increments the local logical time of the process.

2. When process p_i sends a message to process p_j , it piggybacks only the scalar value $D_i[i]$.
3. When process p_i receives a message from p_j with piggybacked value d :

$$D_i[j] = \max(D_i[j], d)$$

This records that the current state of p_i directly depends on event d at process p_j .

Explanation of Figure : Figure shows an execution involving four processes p_1, p_2, p_3 , and p_4 . Each event is labeled with the current dependency vector of that process.



- Initially, when an event occurs at p_1 , its vector becomes

$$[1, 0, 0, 0]^T$$

When p_1 sends a message to p_2 , it piggybacks the scalar $\{1\}$, indicating that p_2 now directly depends on the first event of p_1 .

- Process p_3 later sends a message to p_2 with scalar $\{1\}$. After receiving it, p_2 updates its dependency vector to reflect that its state depends on the first event of p_3 .
- Next, p_3 sends another message carrying scalar $\{2\}$, indicating that the second event of p_3 affects the receiving process.
- Process p_4 sends a message to p_3 carrying $\{1\}$. This indicates that p_3 now directly depends on the first event of p_4 .
- Later, p_3 sends a message to p_2 carrying $\{4\}$, informing p_2 about its direct dependency on the fourth event of p_3 .

Direct vs. Transitive Dependency An important characteristic of this technique is that **only direct dependencies are recorded**.

For example, when:

$$p_4 \rightarrow p_3 \rightarrow p_2$$

process p_2 becomes indirectly dependent on p_4 . However, p_2 is not explicitly informed about this dependency during execution.

The transitive dependency

$$p_4 \rightarrow p_2$$

can only be discovered later by recursively examining the dependency vectors of the events.

Why the last entry of p_2 is 0 The final dependency vector of p_2 is

$$[1, 4, 4, 0]$$

and not

$$[1, 4, 4, 1].$$

This is because the Fowler–Zwaenepoel technique records only **direct dependencies**. Process p_2 receives messages only from process p_1 and process p_3 .

Although process p_3 itself depends on process p_4 , this information is not transmitted to p_2 because only a scalar value representing the sender's local event number is piggybacked in the message.

Therefore p_2 becomes directly dependent on p_3 but not explicitly on p_4 . The transitive dependency

$$p_4 \rightarrow p_3 \rightarrow p_2$$

can be discovered only later through off-line analysis.

Advantages

- Messages carry only a **single scalar value**.
- Communication overhead is greatly reduced.
- Suitable for systems with many processes.

Limitations

- Transitive causal dependencies are not known during execution.
- Determining full causality requires an **off-line analysis**.

Applications This technique is useful in systems where causal relationships are analyzed after execution, such as:

- causal breakpoints
- asynchronous checkpoint recovery
- distributed debugging

22.3 Off-line Computation of Transitive Dependencies

In the Fowler–Zwaenepoel technique, only **direct dependencies** are recorded during execution. However, the complete set of causal (transitive) dependencies of an event can be determined later by recursively combining the direct dependencies of related events.

Basic Idea Suppose an event e_i depends directly on another event e_j . If e_j itself depends on some earlier events, then e_i also indirectly depends on those events.

Thus transitive dependencies can be obtained by recursively examining the direct dependency vectors of the events involved.

Example from last Figure Consider the following dependencies:

- The fourth event of process p_3 depends on the first event of p_4 .
- The fourth event of process p_2 depends on the fourth event of p_3 .

Combining these two direct dependencies, we can deduce the **transitive dependency**:

$$p_4^1 \rightarrow p_3^4 \rightarrow p_2^4$$

Thus the fourth event of process p_2 indirectly depends on the first event of process p_4 .

Observation If an event e_j at process p_j occurs before event e_i at process p_i , then all earlier events at p_j also occur before e_i .

$$e_0, e_1, \dots, e_{j-1} \rightarrow e_i$$

Therefore, it is sufficient to record only the **latest event of process** p_j that happened before e_i .

Dependency Tracking Vector To compute all causal dependencies of an event, a vector called the **Dependency Tracking Vector (DTV)** is used.

$$DTV = [d_1, d_2, \dots, d_n]$$

where d_j represents the latest event of process p_j that causally precedes the event under consideration. Initially,

$$DTV[k] = 0 \quad \forall k \neq i$$

$$DTV[i] = D_i^e[i]$$

where D_i^e is the direct dependency vector of event e_i .

Recursive Dependency Tracking The algorithm then recursively explores dependencies.

For each process j :

- If $D_i^e[j] > DTV[j]$, update

$$DTV[j] = D_i^e[j]$$

- Recursively examine event $e_{DTV[j]}$ on process p_j to include its dependencies.

This process continues until no new dependencies are discovered.

Result After the recursion finishes, the vector DTV contains the complete set of causal dependencies of event e_i .

Thus although only direct dependencies are stored during execution, the full causal history can be reconstructed later using off-line analysis.

22.4 Real-Life Applications of Direct Dependency Tracking

The Fowler–Zwaenepoel technique is useful in systems where tracking full causal history during execution would be too expensive. Instead, only direct dependencies are recorded, and full causality is reconstructed later using off-line analysis.

Distributed Debugging In large distributed applications such as microservices systems, an error observed at one service may be caused by a chain of events in several other services.

For example, in an e-commerce system:

- p_4 (Inventory Service) updates stock.
- p_3 (Order Service) processes the order using the updated stock.
- p_2 (Payment Service) processes the payment after receiving the order confirmation.

If a payment failure occurs at p_2 , the system may need to determine whether it was caused by an earlier inventory update at p_4 . Direct dependencies recorded during execution allow the system to reconstruct the causal chain later during debugging.

Checkpoint Recovery Distributed systems often create checkpoints so that they can recover from failures.

Consider four processes:

- p_1 – authentication service
- p_2 – payment service
- p_3 – order processing
- p_4 – inventory management

If the payment service crashes, the system must determine which checkpoints are consistent with each other.

Using direct dependencies, the system can later compute which events in other services causally affected the payment service before the crash. This helps determine a consistent recovery state.

Causal Breakpoints During distributed debugging, developers may want to stop execution when a specific causal chain occurs.

For example, suppose a developer wants to stop execution when:

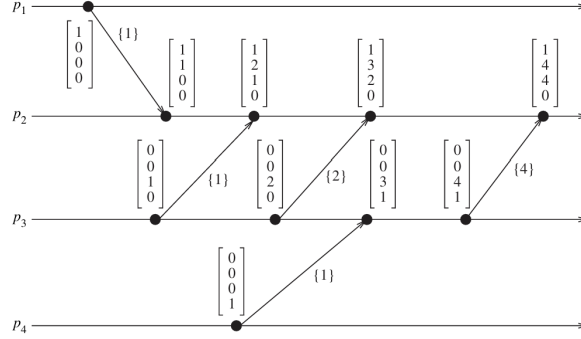
$$\text{Inventory Update} \rightarrow \text{Order Processing} \rightarrow \text{Payment Authorization}$$

The system records direct dependencies during execution. Later, an off-line analysis determines whether such a causal chain actually occurred.

Localized Communication Systems In many distributed systems, communication tends to be localized. For instance, in a cloud application with hundreds of services, each service typically communicates with only a few neighbors.

Instead of sending full vector clocks of size n in every message, the direct dependency technique allows messages to carry only a small scalar value, significantly reducing communication overhead.

22.5 Example of Off-line Dependency Tracking



Consider tracking the causal dependencies of the fourth event at process p_2 . The algorithm is invoked as:

$$DependencyTrack(2, 4)$$

Initialization The Dependency Tracking Vector (DTV) is initialized as:

$$DTV = \langle 0, 4, 0, 0 \rangle$$

This means we begin by assuming the event depends only on the fourth event of process p_2 .

Step 1: Examine Direct Dependencies of $(p_2, 4)$ The direct dependency vector recorded for the fourth event of p_2 is:

$$D_2^4 = \langle 1, 4, 4, 0 \rangle$$

Comparing with DTV , the first entry is larger than the current value. Therefore:

$$DTV \leftarrow \langle 1, 4, 0, 0 \rangle$$

The algorithm now recursively visits event $(p_1, 1)$:

$$VisitEvent(1, 1)$$

Step 2: Visit Event $(p_1, 1)$ The dependency vector for this event is:

$$D_1^1 = \langle 1, 0, 0, 0 \rangle$$

Since none of the entries exceed those already stored in DTV , no update is required and the algorithm returns.

Step 3: Continue Checking ($p_2, 4$) Next, the third entry of D_2^4 is larger than the corresponding entry in DTV .

$$DTV \leftarrow \langle 1, 4, 4, 0 \rangle$$

The algorithm recursively visits event ($p_3, 4$):

$$VisitEvent(3, 4)$$

Step 4: Visit Event ($p_3, 4$) The dependency vector stored for this event is:

$$D_3^4 = \langle 0, 0, 4, 1 \rangle$$

The fourth entry is greater than the current value in DTV . Therefore:

$$DTV \leftarrow \langle 1, 4, 4, 1 \rangle$$

The algorithm now recursively visits event ($p_4, 1$).

Step 5: Visit Event ($p_4, 1$) The dependency vector for this event is:

$$D_4^1 = \langle 1, 0, 0, 0 \rangle$$

No new dependencies are discovered, so the recursion terminates.

Final Result After the algorithm finishes, the Dependency Tracking Vector becomes:

$$DTV = \langle 1, 4, 4, 1 \rangle$$

This indicates that the fourth event of process p_2 causally depends on:

- Event 1 of process p_1
- Event 4 of process p_3
- Event 1 of process p_4

Additionally, the event depends on all events that occurred before these events on their respective processes.

Key Observation This method records only direct dependencies during execution. The complete causal history is reconstructed later through recursive analysis.

Advantages

- Only a single scalar value is piggybacked on messages.
- Communication overhead is significantly reduced.

Limitations

- Full causality information is not available during execution.
- Transitive dependencies must be computed off-line.
- If many events occur, storing dependency history may become expensive.

23 Jard–Jourdan’s Adaptive Technique

The Fowler–Zwaenepoel technique records only direct dependencies and reconstructs transitive dependencies off-line. However, this approach has two main limitations:

- A process must record an event immediately after receiving a message and before sending another message; otherwise causal dependencies may be lost.
- If events occur frequently, the system must store a large history of dependency vectors.

To address these issues, **Jard and Jourdan** proposed an **adaptive observation technique**. The key idea is to record dependency information only at selected events called **observed events**. These observed events store the dependency information accumulated so far and then reset the tracking data.

Thus, future events can reconstruct their dependencies by referring to the recorded information of earlier observed events.

Key Idea

When an observed event e records its dependencies:

- The process stores the dependency information of e .
- The dependency tracking information at that process is reset.
- Future messages propagate only the activity that occurred *after* event e .

Therefore, the next observed event can combine its own information with that recorded at earlier observed events to reconstruct full causal dependencies.

Pseudo-Direct Dependency Relation

Jard–Jourdan introduced a relation called **pseudo-direct dependency**, denoted by $\prec$.

For events e_j at process p_j and e_i at process p_i ,

$$e_j \prec e_i$$

if there exists a chain of message transmissions that starts after e_j and ends before e_i with **no observed event** on the path.

This relation is called *pseudo-direct* because there may exist many intermediate unobserved events between e_j and e_i , but they are treated as if the dependency were direct.

Partial Vector Clock

Each process p_i maintains a **partial vector clock**

$$p_vt_i = \{(j, v)\}$$

where the tuple (j, v) indicates that process p_i is pseudo-dependent on event number v of process p_j . Initially,

$$p_vt_i = \{(i, 0)\}$$

Observed Event Rule

When an event at process p_i is observed:

$$e_vt_i = p_vt_i$$

The dependency information is stored in e_vt_i , and the partial vector clock is reset:

$$p_vt_i = \{(i, v + 1)\}$$

This ensures that future dependency tracking begins after the observed event.

Message Send Rule

When process p_j sends a message to process p_i , it piggybacks its partial vector clock:

$$p_vt_j$$

Message Receive Rule

When process p_i receives a message containing a partial vector clock p_vt :

- Entries present only in p_vt are added to p_vt_i .
- Entries present only in p_vt_i are retained.
- For entries present in both, the maximum timestamp is kept.

Thus p_vt_i becomes the union of both clocks.

Advantages

- Reduces the amount of dependency history stored.
- Allows adaptive observation of events.
- Avoids excessive storage required in direct-dependency tracking.

Limitation

Although it reduces storage overhead, the method still cannot determine all causal dependencies instantly during execution; some reconstruction is still required.

23.1 Illustration of Jard–Jourdan Adaptive Technique

Figure illustrates the Jard–Jourdan adaptive dependency tracking mechanism using five processes p_1, p_2, p_3, p_4 , and p_5 .

Each process maintains a **partial vector clock** p_vt_i , which stores pseudo-direct dependencies in the form of tuples (j, v) , indicating dependency on event number v at process p_j .

Observed events store their dependencies in e_vt_i , while unobserved events temporarily accumulate dependency information in p_vt_i .

Initialization

Initially every process starts with

$$p_vt_i = \{(i, 0)\}$$

indicating that the process depends only on its own initial event.

Observed Event Mechanism

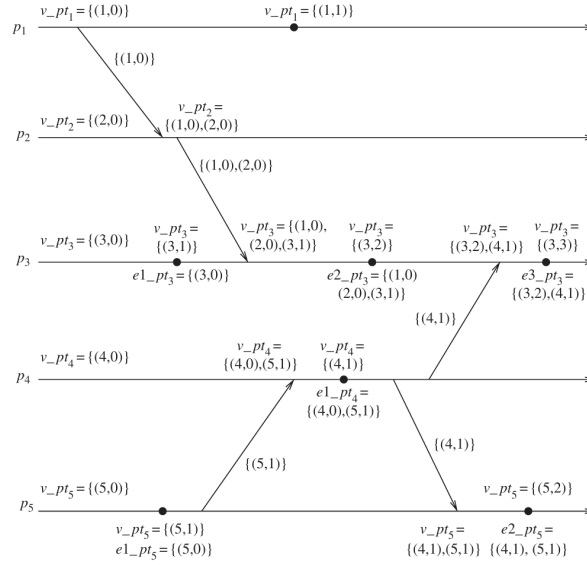
When an event at process p_i is observed:

$$e_vt_i = p_vt_i$$

The partial vector clock is then reset and incremented:

$$p_vt_i = \{(i, v + 1)\}$$

Thus the observed event stores the dependency history and the process begins tracking new dependencies after that point.



Example: Events at Process p_3

Consider the behavior of process p_3 in the figure.

- Initially,

$$p_vt_3 = \{(3, 0)\}$$

When the first observed event occurs at p_3 , the dependency list is stored as

$$e1_pt_3 = \{(3, 0)\}$$

and the partial vector clock becomes

$$p_vt_3 = \{(3, 1)\}$$

- Later, p_3 receives a message from process p_2 containing

$$\{(1, 0), (2, 0)\}$$

These dependencies are merged into its list:

$$p_vt_3 = \{(1, 0), (2, 0), (3, 1)\}$$

3. When the second observed event occurs at p_3 , the list is stored as

$$e2_pt_3 = \{(1, 0), (2, 0), (3, 1)\}$$

and the partial vector clock is reset to

$$p_vt_3 = \{(3, 2)\}$$

4. Later, after receiving information from p_4 :

$$\{(4, 1)\}$$

the list becomes

$$p_vt_3 = \{(3, 2), (4, 1)\}$$

5. When the third observed event occurs, the dependencies are stored as

$$e3_pt_3 = \{(3, 2), (4, 1)\}$$

Why both (3,2) and (4,1) appear After the second observed event at p_3 , the partial vector clock is reset to

$$p_vt_3 = \{(3, 2)\}$$

which represents the dependency on the latest local event of p_3 .

When a message is received from p_4 carrying the timestamp $\{(4, 1)\}$, the partial vector clock is updated by taking the union of the existing dependencies and the received dependencies.

$$p_vt_3 = \{(3, 2)\} \cup \{(4, 1)\}$$

$$p_vt_3 = \{(3, 2), (4, 1)\}$$

Thus the next observed event at p_3 depends on both the previous local event (3,2) and the message received from p_4 (4,1).

Pseudo-Direct Dependency

Event $e2_pt_3$ is directly dependent on:

$$(2, 0), (3, 1)$$

but it is **pseudo-directly dependent** on:

$$(1, 0)$$

because the dependency passes through p_2 without another observed event on the path.

Thus some dependencies are recorded indirectly and can be resolved later by examining previously recorded observed events.

Recovering Transitive Dependencies

The complete causal dependencies of an observed event can be reconstructed by recursively examining the observed events stored in the dependency lists.

However, these dependencies cannot be determined during execution and must be computed from the event logs.

Advantages

- Reduces message size by sending only partial dependency lists.
- Avoids storing long dependency histories.
- Allows adaptive observation of events.

Controlling Message Size

If the partial vector clock becomes too large, a **dummy observed event** can be introduced.

If the list size reaches a threshold k , the system records a dummy observed event and resets the list. This allows flexible control over message sizes in the system.

23.2 Matrix Time

Matrix clocks extend vector clocks by tracking not only what a process knows about other processes, but also what it believes other processes know.

Each process p_i maintains an $n \times n$ matrix clock:

$$mt_i[1..n][1..n]$$

Meaning of Entries

- $mt_i[i, i]$ Local logical clock of process p_i .
- $mt_i[i, j]$ Latest knowledge that p_i has about the local clock of p_j . Row i is identical to the vector clock of p_i .
- $mt_i[j, k]$ What p_i believes that process p_j knows about the local logical clock of process p_k .

Clock Update Rules R1: Local Event

Before executing an event:

$$mt_i[i, i] = mt_i[i, i] + 1$$

R2: Message Receive

When process p_i receives a message from p_j with matrix timestamp mt :

1. Update its vector row:

$$mt_i[i, k] = \max(mt_i[i, k], mt_j[k])$$

2. Update remaining matrix entries:

$$mt_i[k, l] = \max(mt_i[k, l], mt[k, l])$$

3. Increment local clock (rule R1).

Interpretation Matrix clocks provide second-level knowledge of the system. For example,

$$mt_i[j, k]$$

represents what process p_i believes that process p_j knows about process p_k .

Important Property If

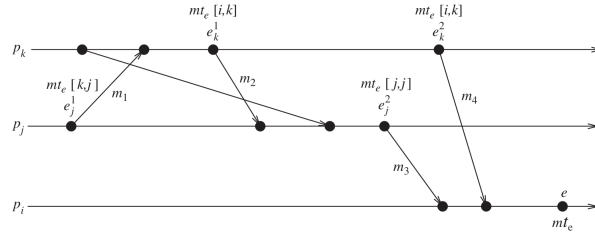
$$\min_k mt_i[k, l] \geq t$$

then process p_i knows that every process p_k knows that process p_l has reached time t .

Applications Matrix clocks are used in distributed systems such as:

- replicated databases
- distributed garbage collection
- detecting obsolete updates

Explanation of Figure: Evolution of Matrix Time The figure shows three processes p_k , p_j , and p_i exchanging messages m_1, m_2, m_3 , and m_4 .



- Event e_j^1 occurs at process p_j and message m_1 is sent to p_k . Thus p_k learns about the progress of p_j .
- Later event e_k^1 occurs at p_k , after which message m_2 is sent to p_j . This allows p_j to learn about the progress of p_k .
- Event e_j^2 occurs at p_j and message m_3 is sent to p_i . Therefore p_i learns about p_j .
- Event e_k^2 occurs at p_k and message m_4 is sent to p_i , allowing p_i to learn about p_k .

Let e be an event at process p_i and let mt_e be its matrix timestamp.

$$mt_e[i, k] = x_k^2$$

indicates that the latest event of p_k known to p_i is e_k^2 .

$$mt_e[i, j] = x_j^2$$

indicates that the latest event of p_j known to p_i is e_j^2 .

$$mt_e[j, k] = x_k^1$$

indicates that according to p_i , process p_j only knew about event e_k^1 at p_k .

$$mt_e[k, j] = x_j^1$$

indicates that according to p_i , process p_k only knew about event e_j^1 at p_j .

Thus matrix clocks capture not only the knowledge of a process about others, but also what a process believes other processes know about the system.